

The Collections Framework

- The *Collections Framework* is one of Java's most powerful subsystems present in **,java.util** package.
- Collections Framework is a sophisticated hierarchy of interfaces and classes that provide SOTA technology for managing groups of objects(standardizes the way).
- Collections were not part of the original Java release, but were added by J2SE 1.2.
- Prior to the Collections Framework, Java provided ad hoc classes such as **Dictionary**, **Vector**, **Stack**, and **Properties** to store and manipulate groups of objects.
 - They were designed **independently**, not as part of a unified system
 - Each class had **different methods and usage styles**(Ex:Using a Vector was different from using Properties)
 - There was **no common interface or structure**
 - It was **hard to extend or reuse** them in a consistent way

Objectives:

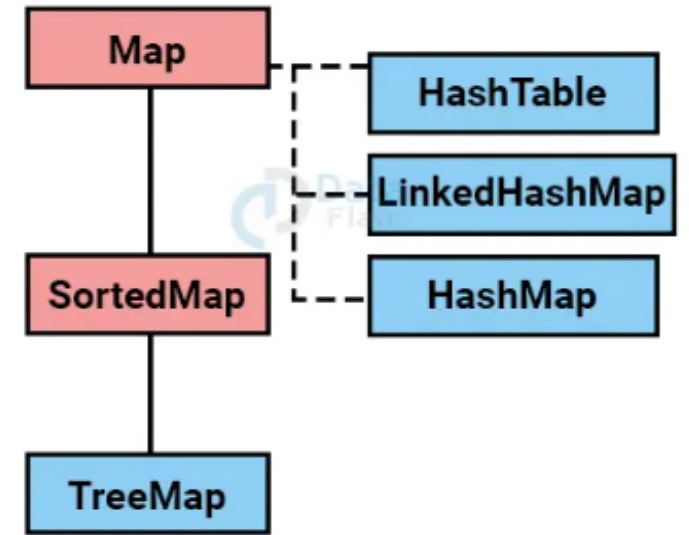
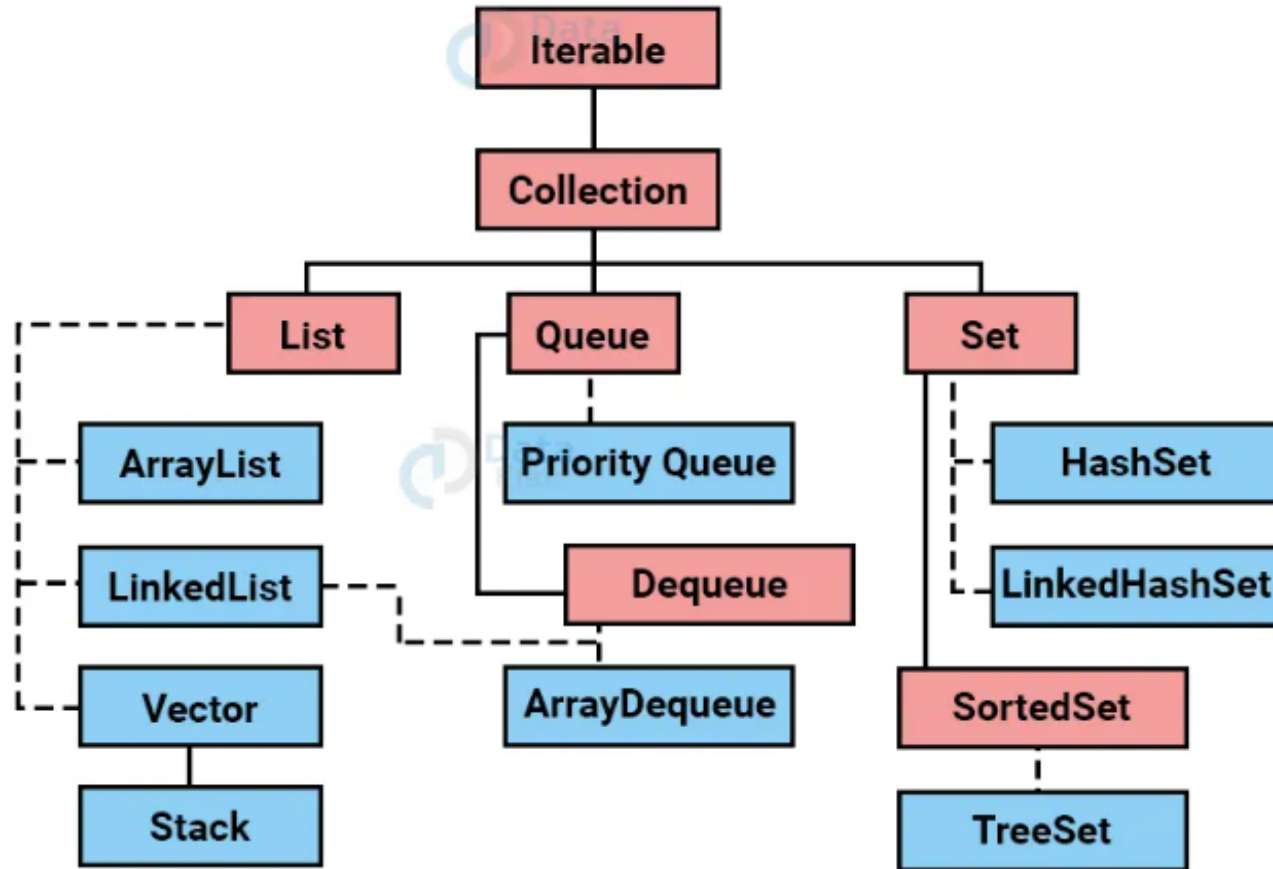
- It provides **high performance**, with efficient built-in data structures like dynamic arrays, linked lists, trees, and hash tables, so no need to implement them from scratch.
- It ensures **uniformity and interoperability**, allowing different collections to be used in a similar way.
- It is **easy to extend and customize**, as it is based on standard interfaces.
- It offers **ready-made implementations** (e.g., LinkedList, HashSet, TreeSet) and also supports creating custom collections.
- It includes **special and partial implementations** to simplify development.
- It allows **integration of arrays** with the Collections Framework.

- Collections use **common algorithms and iterators** to ensure consistent and easy data manipulation and traversal and **map interfaces and classes**.
- Algorithms in the Collections Framework are provided as static methods in the Collections class.
- They offer a standard way to perform operations (like sorting, searching) on collections, so individual classes don't need to implement them separately.
- The Iterator interface provides a uniform way to access elements one by one in any collection.
- It helps in traversing (enumerating) collection elements in a consistent manner. Because all collections use iterators, similar code can work for different collection types (like List, Set, etc.).
- In JDK 8, a new type called Spliterator was introduced for advanced iteration.

A Map stores data as key/value pairs.

- Maps are part of the framework but are not considered true collections.
- However, you can get a collection view of a map (like set of keys, values, or entries).
- This allows you to process map data using collection methods.

Hierarchy of Collection Framework in Java



The Collection Interfaces

Interface	Description
Collection	Enables you to work with groups of objects; it is at the top of the collections hierarchy.
Deque	Extends Queue to handle a double-ended queue.
List	Extends Collection to handle sequences (lists of objects).
NavigableSet	Extends SortedSet to handle retrieval of elements based on closest-match searches.
Queue	Extends Collection to handle special types of lists in which elements are removed only from the head.
Set	Extends Collection to handle sets, which must contain unique elements.
SortedSet	Extends Set to handle sorted sets.

- The Collections Framework also uses supporting interfaces like **Comparator**, **Iterator**, **ListIterator**, **RandomAccess**, and (from JDK 8) **Spliterator**.
- **Comparator** is used to **define how objects are compared** (e.g., for sorting).
- **Iterator**, **ListIterator**, and **Spliterator** are used to **traverse elements** in a collection.
- **RandomAccess** indicates that a list supports **fast, direct access** to elements (like arrays).
- These interfaces help with **comparing, accessing, and traversing collection elements efficiently**.

- Some methods in collection interfaces are **optional**, mainly those that **modify the collection**.
- Collections that support these methods are called **modifiable collections**.
- Collections that do not allow changes are called **unmodifiable collections**.
- If you try to modify an unmodifiable collection, it throws an **UnsupportedOperationException**.
- All **built-in collections** are **modifiable by default**.

The Collection Interface

- The **Collection** interface is the foundation upon which the Collections Framework is built because it must be implemented by any class that defines a collection.
- **Collection** is a generic interface that has this declaration:

```
interface Collection<E>
```

E :The type of objects that the collection will hold.

Collection extends the **Iterable** interface. This means that all collections can be cycled through by use of the for each style **for** loop. (Recall that only classes that implement **Iterable** can be cycled through by the **for**.)

- The **Collection interface** defines the **core methods** that all collections must implement.
- Understanding these methods is essential to understand the **entire Collections Framework**.

Method	Description
boolean add(E <i>obj</i>)	Adds <i>obj</i> to the invoking collection. Returns true if <i>obj</i> was added to the collection. Returns false if <i>obj</i> is already a member of the collection and the collection does not allow duplicates.
boolean addAll(Collection<? extends E> <i>c</i>)	Adds all the elements of <i>c</i> to the invoking collection. Returns true if the collection changed (i.e., the elements were added). Otherwise, returns false .
void clear()	Removes all elements from the invoking collection.
boolean contains(Object <i>obj</i>)	Returns true if <i>obj</i> is an element of the invoking collection. Otherwise, returns false .
boolean containsAll(Collection<?> <i>c</i>)	Returns true if the invoking collection contains all elements of <i>c</i> . Otherwise, returns false .
boolean equals(Object <i>obj</i>)	Returns true if the invoking collection and <i>obj</i> are equal. Otherwise, returns false .
int hashCode()	Returns the hash code for the invoking collection.
boolean isEmpty()	Returns true if the invoking collection is empty. Otherwise, returns false .
Iterator<E> iterator()	Returns an iterator for the invoking collection.
default Stream<E> parallelStream()	Returns a stream that uses the invoking collection as its source for elements. If possible, the stream supports parallel operations. (Added by JDK 8.)
boolean remove(Object <i>obj</i>)	Removes one instance of <i>obj</i> from the invoking collection. Returns true if the element was removed. Otherwise, returns false .
boolean removeAll(Collection<?> <i>c</i>)	Removes all elements of <i>c</i> from the invoking collection. Returns true if the collection changed (i.e., elements were removed). Otherwise, returns false .
default boolean removeIf(Predicate<? super E> <i>predicate</i>)	Removes from the invoking collection those elements that satisfy the condition specified by <i>predicate</i> . (Added by JDK 8.)

- Some methods may throw exceptions, especially when operations are not supported or invalid:
 - **UnsupportedOperationException** → when modification is not allowed
 - **ClassCastException** → when adding incompatible object types
 - **NullPointerException** → when null is not allowed but used
 - **IllegalArgumentException** → when an invalid argument is passed
 - **IllegalStateException** → when adding to a full fixed-size collection

The List Interface

- The **List** interface extends **Collection** and declares the behavior of a collection that stores a sequence of elements. Elements can be inserted or accessed by their position in the list, using a zero-based index. A list may contain duplicate elements.

Declaration of **List** : `interface List<E>`

E : The type of objects that the list will hold.

Exceptions

- **UnsupportedOperationException** if the list cannot be modified
- **ClassCastException**: when one object is incompatible with another
- **IndexOutOfBoundsException** if an invalid index is used.
- A **NullPointerException** :if an attempt is made to store a **null** object and **null** elements to list.
- An **IllegalArgumentException** :if an invalid argument is used.

Method	Description
<code>void add(int <i>index</i>, E <i>obj</i>)</code>	Inserts <i>obj</i> into the invoking list at the index passed in <i>index</i> . Any preexisting elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten.
<code>boolean addAll(int <i>index</i>, Collection<? extends E> <i>c</i>)</code>	Inserts all elements of <i>c</i> into the invoking list at the index passed in <i>index</i> . Any preexisting elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten. Returns <code>true</code> if the invoking list changes and returns <code>false</code> otherwise.
<code>E get(int <i>index</i>)</code>	Returns the object stored at the specified index within the invoking collection.
<code>int indexOf(Object <i>obj</i>)</code>	Returns the index of the first instance of <i>obj</i> in the invoking list. If <i>obj</i> is not an element of the list, <code>-1</code> is returned.
<code>int lastIndexOf(Object <i>obj</i>)</code>	Returns the index of the last instance of <i>obj</i> in the invoking list. If <i>obj</i> is not an element of the list, <code>-1</code> is returned.
<code>ListIterator<E> listIterator()</code>	Returns an iterator to the start of the invoking list.
<code>ListIterator<E> listIterator(int <i>index</i>)</code>	Returns an iterator to the invoking list that begins at the specified <i>index</i> .
<code>E remove(int <i>index</i>)</code>	Removes the element at position <i>index</i> from the invoking list and returns the deleted element. The resulting list is compacted. That is, the indexes of subsequent elements are decremented by one.
default void <code>replaceAll(UnaryOperator<E> <i>opToApply</i>)</code>	Updates each element in the list with the value obtained from the <i>opToApply</i> function. (Added by JDK 8.)
<code>E set(int <i>index</i>, E <i>obj</i>)</code>	Assigns <i>obj</i> to the location specified by <i>index</i> within the invoking list. Returns the old value.
default void <code>sort(Comparator<? super E> <i>comp</i>)</code>	Sorts the list using the comparator specified by <i>comp</i> . (Added by JDK 8.)
<code>List<E> subList(int <i>start</i>, int <i>end</i>)</code>	Returns a list that includes elements from <i>start</i> to <i>end</i> -1 in the invoking list. Elements in the returned list are also referenced by the invoking object.

- To the versions of **add()** and **addAll()** defined by **Collection**, **List** adds the methods **add(int, E)** and **addAll(int, Collection)**: These methods insert elements at the specified index.
- The semantics of **add(E)** and **addAll(Collection)** defined by **Collection** are changed by **List** so that they add elements to the end of the list.
- You can modify each element in the collection by using **replaceAll()**.
- To obtain the object stored at a specific location, call **get()** with the index of the object.
- To assign a value to an element in the list, call **set()**, specifying the index of the object to be changed.
- To find the index of an object, use **indexOf()** or **lastIndexOf()**.
- You can obtain a sublist of a list by calling **subList()**, specifying the beginning and ending indexes of the sublist and makes list processing quite convenient.
- Sort a list is with the **sort()** method defined by **List**.

The Set Interface

- The **Set** interface defines a set. It extends **Collection** and specifies the behavior of a collection that does not allow duplicate elements. Therefore, the **add()** method returns **false** if an attempt is made to add duplicate elements to a set.
- It does not specify any additional methods of its own. **Set** is a generic interface that has this declaration:

```
interface Set<E>
```

E :the type of objects that the set will hold

The Queue interface

- The Queue interface extends Collection and declares the behavior of a queue
- first-in, first-out list. However,
- There are types of queues in which the ordering is based upon other criteria called Priority Queue.
- Queue is a generic interface that has this declaration:
 interface Queue<E>
- E :The type of objects that the queue will hold.

Method	Description
E element()	Returns the element at the head of the queue. The element is not removed. It throws NoSuchElementException if the queue is empty.
boolean offer(E <i>obj</i>)	Attempts to add <i>obj</i> to the queue. Returns true if <i>obj</i> was added and false otherwise.
E peek()	Returns the element at the head of the queue. It returns null if the queue is empty. The element is not removed.
E poll()	Returns the element at the head of the queue, removing the element in the process. It returns null if the queue is empty.
E remove()	Removes the element at the head of the queue, returning the element in the process. It throws NoSuchElementException if the queue is empty.

The Deque Interface

- The Deque interface extends Queue and declares the behavior of a double-ended queue.
- Double-ended queues can function as standard, first-in, first-out queues or as last-in, first-out stacks.
- Deque is a generic interface that has this declaration:

```
interface Deque<E>
```

E : the type of objects that the deque will hold.

In addition to the methods that it inherits from Queue, Deque adds few more methods, few are shown in next table.

Method	Description
void addFirst(E <i>obj</i>)	Adds <i>obj</i> to the head of the deque. Throws an IllegalStateException if a capacity-restricted deque is out of space.
void addLast(E <i>obj</i>)	Adds <i>obj</i> to the tail of the deque. Throws an IllegalStateException if a capacity-restricted deque is out of space.
Iterator<E> descendingIterator()	Returns an iterator that moves from the tail to the head of the deque. In other words, it returns a reverse iterator.
E getFirst()	Returns the first element in the deque. The object is not removed from the deque. It throws NoSuchElementException if the deque is empty.
E getLast()	Returns the last element in the deque. The object is not removed from the deque. It throws NoSuchElementException if the deque is empty.
boolean offerFirst(E <i>obj</i>)	Attempts to add <i>obj</i> to the head of the deque. Returns true if <i>obj</i> was added and false otherwise. Therefore, this method returns false when an attempt is made to add <i>obj</i> to a full, capacity-restricted deque.
boolean offerLast(E <i>obj</i>)	Attempts to add <i>obj</i> to the tail of the deque. Returns true if <i>obj</i> was added and false otherwise.
E peekFirst()	Returns the element at the head of the deque. It returns null if the deque is empty. The object is not removed.
E peekLast()	Returns the element at the tail of the deque. It returns null if the deque is empty. The object is not removed.
E pollFirst()	Returns the element at the head of the deque, removing the element in the process. It returns null if the deque is empty.

Collection Classes

- Standard classes that implement interfaces.
- Some of the classes provide full implementations that can be used as-is and Others are abstract, that are used as starting points for creating concrete collections.

The core collection classes are summarized in the following table:

Class	Description
<code>AbstractCollection</code>	Implements most of the Collection interface.
<code>AbstractList</code>	Extends AbstractCollection and implements most of the List interface.
<code>AbstractQueue</code>	Extends AbstractCollection and implements parts of the Queue interface.
<code>AbstractSequentialList</code>	Extends AbstractList for use by a collection that uses sequential rather than random access of its elements.
<code>LinkedList</code>	Implements a linked list by extending AbstractSequentialList .
<code>ArrayList</code>	Implements a dynamic array by extending AbstractList .
<code>ArrayDeque</code>	Implements a dynamic double-ended queue by extending AbstractCollection and implementing the Deque interface.
<code>AbstractSet</code>	Extends AbstractCollection and implements most of the Set interface.
<code>EnumSet</code>	Extends AbstractSet for use with enum elements.
<code>HashSet</code>	Extends AbstractSet for use with a hash table.
<code>LinkedHashSet</code>	Extends HashSet to allow insertion-order iterations.
<code>PriorityQueue</code>	Extends AbstractQueue to support a priority-based queue.
<code>TreeSet</code>	Implements a set stored in a tree. Extends AbstractSet .

Collection Classes to be discussed

Collection	List	Set	Queue	Deque	Map
	ArrayList	HashSet	PriorityQueue	ArrayDeque	HashMap
	LinkedList	LinkedHashSet			
	Vector	TreeSet			
	Stack				

List Implementation Classes

◆ Main Classes that Implement List

✓ 1. ArrayList

- Most commonly used
- Uses **dynamic array**
- Fast for **access (get)**

⟨/⟩ Java

```
ArrayList<Integer> list = new ArrayList<>();
```

✓ 2. LinkedList

- Uses **doubly linked list**
- Fast for **insertion/deletion**

⟨/⟩ Java

```
LinkedList<Integer> list = new LinkedList<>();
```

✓ 3. Vector

- Legacy class (older)
- **Synchronized** (thread-safe)

⟨/⟩ Java

```
Vector<Integer> list = new Vector<>();
```

✓ 4. Stack

- Subclass of Vector
- Works as **LIFO (stack)**

⟨/⟩ Java

```
Stack<Integer> stack = new Stack<>();
```

◆ Other Less Common / Special Cases

✓ 5. CopyOnWriteArrayList (java.util.concurrent)

- Thread-safe
- Used in **concurrent applications**

⟨/⟩ Java

```
CopyOnWriteArrayList<Integer> list = new CopyOnWriteArrayList<>();
```

✓ 6. AbstractList (Indirect)

- Abstract class
- Used to create custom list implementations

The ArrayList Class

- The **ArrayList** class extends **AbstractList** and implements the **List** interface.
- **ArrayList** is a generic class that has this declaration:

```
class ArrayList<E>
```

E : the type of objects that the list will hold.

- **ArrayList** supports dynamic arrays that can grow as needed.
- In Java, standard arrays are of a fixed length an **ArrayList** is a variable-length array of object references.
- Array lists are created with an initial size. When this size is exceeded, the collection is automatically enlarged. When objects are removed, the array can be shrunk.

Constructors of **ArrayList**

- `ArrayList( )` // builds an empty array list
- `ArrayList(Collection c)` // builds an array list that is initialized
with the elements of the collection c
- `ArrayList(int capacity)` // builds an array list that has the specified initial *capacity*.

An ArrayList can store **objects only** (not primitive data types directly). The type of objects it can store depends on how you declare it.

- **1. Specific Type (Using Generics)**

- If you declare with a type, it stores **only that type of objects**:
- `ArrayList<String> list = new ArrayList<>();` //Stores only String objects

Other examples:

- `ArrayList<Integer> nums = new ArrayList<>();` // Integers
`ArrayList<Double> values = new ArrayList<>();` // Doubles
`ArrayList<Character> chars = new ArrayList<>();` // Characters
- Note: Primitive types like int, double, char are **not allowed directly** ,Use wrapper classes (Integer, Double, Character)

2. User-Defined Objects

- You can store objects of your own classes:
- ```
class Student {
 int id;
 String name;
}
```

`ArrayList<Student> students = new ArrayList<>();` //Stores Student objects

### 3. Mixed Objects (Not Recommended)

If you don't use generics:

- `ArrayList list = new ArrayList();`

Can store **any type of object**:

- `list.add("Hello");`  
`list.add(10);`  
`list.add(3.14);` //But this is **unsafe** and not recommended because it can cause runtime errors.

### 4. Objects of Parent Type

You can store objects of a class and its subclasses:

- `ArrayList<Object> list = new ArrayList<>();`

Can store anything (since all classes extend Object)

Difference between 3<sup>rd</sup> and 4<sup>th</sup> shown in next slide but by taking LinkedList class (same for ArrayList also)

With Linked List, which is discussed in next slide....It is also same for ArrayList

## 1. With Generics

</> Java

```
LinkedList<Object> list = new LinkedList<>();
```

- Uses **generics**
- The list is restricted to **Object type (and its subclasses)**
- **Type-safe at compile time**
- No warning from compiler

✓ Example:

</> Java

```
list.add("Hello");
list.add(10);
list.add(3.14);
```

👉 Still allows mixed types, but in a **controlled, type-safe way**

## 2. Without Generics (Raw Type)

</> Java

```
LinkedList list = new LinkedList();
```

- This is a **raw type**
- Can store **any type of object**
- **No type checking at compile time**
- Compiler gives a **warning**

✓ Example:



</> Java

```
LinkedList list = new LinkedList();
```

- This is a **raw type**
- Can store **any type of object**
- **No type checking at compile time**
- Compiler gives a **warning**

✓ Example:

</> Java

```
list.add("Hello");
list.add(10);
list.add(3.14);
```

## 🔥 Key Differences

| Feature               | LinkedList<Object> | LinkedList (raw) |
|-----------------------|--------------------|------------------|
| Type safety           | ✓ Yes              | ✗ No             |
| Compile-time checking | ✓ Yes              | ✗ No             |
| Compiler warning      | ✗ No               | ⚠ Yes            |
| Recommended           | ✓ Yes              | ✗ No             |



# LinkedList Class

- The LinkedList class extends AbstractSequentialList and implements the List, Deque, and Queue interfaces.
- It uses a **doubly linked list** data structure to store elements.
- LinkedList does not directly implement Collection, but it implements interfaces that extend Collection.

class LinkedList<E>

**E** :Type of objects that the list will hold.

**LinkedList** has the two constructors

- LinkedList( ) // The first constructor builds an empty linked list.
- LinkedList(Collection *c*) builds a linked list that is initialized with the elements of the collection *c*.
- Because **LinkedList implements both List and Deque interfaces**, it **inherits all the methods defined in both interfaces** (and also from Collection, since both extend it).

## Common Methods

### ◆ Key Characteristics

- Maintains **insertion order**
- Allows **duplicate elements**
- Allows **null values**
- Not synchronized (not thread-safe)
- Provides **fast insertion and deletion**
- Slower for **random access** (index-based access)

### ✓ Adding Elements

```
</> Java

add(E e)
add(int index, E element)
addFirst(E e)
addLast(E e)
```

### ✓ Removing Elements

```
</> Java

remove()
remove(int index)
remove(Object o)
removeFirst()
removeLast()
```

### ◆ Internal Working

- Each element is stored in a **node**
- Each node contains:
  - Data
  - Reference to **previous node**
  - Reference to **next node**

### ✓ Accessing Elements

```
</> Java

get(int index)
getFirst()
getLast()
```

### ✓ Queue Methods

```
</> Java

offer(E e)
poll()
peek()
```

### ✓ Stack Methods

```
</> Java

push(E e)
pop()
```

### ◆ Advantages

- Efficient **insertion and deletion**
- Can be used as **List, Queue, and Stack**
- No need to shift elements

### ◆ Disadvantages

- **Slow access** (no direct indexing)
- **More memory usage** (extra pointers)
- Not suitable for **frequent searching**

### ◆ When to Use

Use `LinkedList` when:

- You frequently **insert/delete elements**
- You need **queue or deque operations**
- You don't require fast random access

## From List interface

You can use methods like:

- `add(E e)`
- `add(int index, E element)`
- `get(int index)`
- `set(int index, E element)`
- `remove(int index)`
- `indexOf()`, `lastIndexOf()`

## From Deque interface

You can use methods like:

- `addFirst()`, `addLast()`
- `removeFirst()`, `removeLast()`
- `getFirst()`, `getLast()`
- `offer()`, `poll()`, `peek()`
- `push()`, `pop()`

You're not getting "two separate objects."

It's **one LinkedList object**, but it supports **multiple behavi**

- Works like a **List**
- Works like a **Queue**
- Works like a **Stack**

### ♦ One Subtle Detail (Important for Exams)

Access depends on reference type:

Java

```
List<Integer> list = new LinkedList<>();
list.add(10); // ✅ allowed
// list.addFirst(5); ❌ not allowed (List reference)
```

Java

```
Deque<Integer> list = new LinkedList<>();
list.addFirst(5); // ✅ allowed
```



So:

- Object type = `LinkedList`
- Accessible methods depend on reference type

```
// Demonstrate LinkedList.
import java.util.*;

class LinkedListDemo {
 public static void main(String args[]) {
 // Create a linked list.
 LinkedList<String> ll = new LinkedList<String>();

 // Add elements to the linked list.
 ll.add("F");
 ll.add("B");
 ll.add("D");
 ll.add("E");
 ll.add("C");
 ll.addLast("Z");
 ll.addFirst("A");

 ll.add(1, "A2");

 System.out.println("Original contents of ll: " + ll);

 // Remove elements from the linked list.
 ll.remove("F");
 ll.remove(2);

 System.out.println("Contents of ll after deletion: ")

 // Remove first and last elements.
 ll.removeFirst();
 ll.removeLast();

 System.out.println("ll after deleting first and last: "
 + ll);

 // Get and set a value.

 String val = ll.get(2);
 ll.set(2, val + " Changed");

 System.out.println("ll after change: " + ll);
 }
}
```

The output from this program is shown here:

```
Original contents of ll: [A, A2, F, B, D, E, C, Z]
Contents of ll after deletion: [A, A2, D, E, C, Z]
ll after deleting first and last: [A2, D, E, C]
ll after change: [A2, D, E Changed, C]
```

# HashSet

- HashSet extends AbstractSet and implements the Set interface. It creates a collection that uses a hash table for storage.
- HashSet is a generic class that has this declaration:

```
class HashSet<E>
```

E : The type of objects that the set will hold.

- A hash table stores information by using a mechanism called hashing. In hashing, the informational content of a key is used to determine a unique value, called its hash code i.e index at which the data associated with the key is stored. The transformation of the key into its hash code is performed automatically—you never see the hash code itself. Also, your code can't directly index the hash table.
- The advantage of hashing is that it allows the execution time of add( ), contains( ), remove( ), and size( ) to remain constant even for large sets.

The following constructors are defined:

- HashSet( )
- HashSet(Collection<? extends E> c)
- HashSet(int capacity)
- HashSet(int capacity, float fillRatio)

- HashSet does not define any additional methods beyond those provided by its superclasses and interfaces.
- It is important to note that HashSet does not guarantee the order of its elements, because the process of hashing doesn't usually lend itself to the creation of sorted sets.
- If you need sorted storage, then another collection, such as TreeSet, is a better choice.

```
// Demonstrate HashSet.
import java.util.*;

class HashSetDemo {
 public static void main(String args[]) {
 // Create a hash set.
 HashSet<String> hs = new HashSet<String>();

 // Add elements to the hash set.
 hs.add("Beta");
 hs.add("Alpha");
 hs.add("Eta");
 hs.add("Gamma");
 hs.add("Epsilon");
 hs.add("Omega");

 System.out.println(hs);
 }
}
```

The following is the output from this program:

```
[Gamma, Eta, Alpha, Epsilon, Omega, Beta]
```

# LinkedHashSet

- The **LinkedHashSet** class extends **HashSet** and adds no members of its own. It is a generic class that has this declaration:

```
class LinkedHashSet<E>
```

E :Type of objects that the set will hold. Its constructors parallel those in **HashSet**.

- **LinkedHashSet**, try substituting **LinkedHashSet** for **HashSet** in the preceding program.The output will be in the previous program
- [Beta, Alpha, Eta, Gamma, Epsilon, Omega]



# The TreeSet Class

- **TreeSet** extends **AbstractSet** and implements the **NavigableSet** interface. It creates a collection that uses a tree for storage. Objects are stored in sorted, ascending order.
- Access and retrieval times are quite fast, which makes **TreeSet** an excellent choice when storing large amounts of sorted information that must be found quickly.
- **TreeSet** is a generic class that has this declaration:

```
class TreeSet<E>
```

- **E** :The type of objects that the set will hold.

## Constructors:

- `TreeSet( )`
- `TreeSet(Collection<? extends E> c)`
- `TreeSet(Comparator<? super E> comp)`
- `TreeSet(SortedSet<E> ss)`

```
// Demonstrate TreeSet.
import java.util.*;

class TreeSetDemo {
 public static void main(String args[]) {
 // Create a tree set.
 TreeSet<String> ts = new TreeSet<String>();

 // Add elements to the tree set.
 ts.add("C");
 ts.add("A");
 ts.add("B");
 ts.add("E");
 ts.add("F");
 ts.add("D");

 System.out.println(ts);
 }
}
```

# The PriorityQueue Class

- **PriorityQueue** extends **AbstractQueue** and implements the **Queue** interface. It creates a queue that is prioritized based on the queue's comparator.
- **PriorityQueue** is a generic class that has this declaration:  
class PriorityQueue<E>  
E: The type of objects stored in the queue. **PriorityQueues** are dynamic, growing as necessary.

Six constructors shown here:

- PriorityQueue( )
- PriorityQueue(int *capacity*)
- PriorityQueue(Comparator<? super E> *comp*) (Added by JDK 8.)
- PriorityQueue(int *capacity*, Comparator<? super E> *comp*)
- PriorityQueue(Collection<? extends E> *c*)
- PriorityQueue(PriorityQueue<? extends E> *c*)
- PriorityQueue(SortedSet<? extends E> *c*)

# The ArrayDeque Class

- The **ArrayDeque** class extends **AbstractCollection** and implements the **Deque** interface.
- It adds no methods of its own. **ArrayDeque** creates a dynamic array and has no capacity restrictions.
- **ArrayDeque** is a generic class that has this declaration:

```
class ArrayDeque<E>
```

**E** :the type of objects stored in the collection.

Constructors:

- `ArrayDeque( )` // builds an empty deque. Its starting capacity is 16
- `ArrayDeque(int size)` // constructor builds a deque that has the specified initial capacity
- `ArrayDeque(Collection<? extends E> c)` // creates a deque that is initialized with the elements of the collection passed in c. the capacity grows as needed to handle the elements added to the deque

```
// Demonstrate ArrayDeque.
import java.util.*;

class ArrayDequeDemo {
 public static void main(String args[]) {
 // Create an array deque.
 ArrayDeque<String> adq = new ArrayDeque<String>();

 // Use an ArrayDeque like a stack.
 adq.push("A");
 adq.push("B");
 adq.push("D");

 adq.push("E");
 adq.push("F");

 System.out.print("Popping the stack: ");

 while (adq.peek() != null)
 System.out.print(adq.pop() + " ");

 System.out.println();
 }
}
```

The output is shown here:

Popping the stack: F E D B A

# The Legacy Classes and Interfaces

- Early versions of java.util did not have the Collections Framework and used some older (legacy) classes to store objects.
- When the Collections Framework was introduced in J2SE 1.2, those old classes were updated to fit into it.
- However, modern collection classes (like ArrayList, HashMap) are preferred because they are more efficient and consistent.
- Legacy classes are still supported mainly for compatibility with older code.
- The legacy classes defined by **java.util** are shown here:

|            |           |            |       |        |
|------------|-----------|------------|-------|--------|
| Dictionary | Hashtable | Properties | Stack | Vector |
|------------|-----------|------------|-------|--------|

# Vector

- It is a dynamic array similar to ArrayList, but It is **synchronized** (thread-safe)
- It includes many **legacy methods** (older methods)

Later improvements:

- Updated to **extend AbstractList**
- Implements **List interface**
- Supports **generics (from JDK 5)**
- Implements **Iterable**, so it can be used in **for-each loop**

```
class Vector<E>
```

E:The type of element that will be stored.

**Vector** constructors:

- Vector( )
- Vector(int *size*)
- Vector(int *size*, int *incr*)
- Vector(Collection *c*)

- A Vector starts with an initial capacity. When it becomes full, it automatically increases its size by allocating extra space to reduce frequent memory allocations. The growth amount depends on the increment value given during creation; if not specified, the capacity doubles each time.
- Initial capacity is the number of elements a collection (like Vector or ArrayList) can hold when it is first created, before it needs to grow.
- In simple terms, it is the starting size of internal memory.

Example:

```
Vector<Integer> v = new Vector<>(5);
```

- Here, the initial capacity is 5, so the vector can store 5 elements without resizing. When you add the 6th element, it will increase its capacity automatically.
- Initial capacity is user-dependent, but not fixed.
- You can set it yourself when creating the collection.
- If you don't specify it, Java uses a default value.
- It is not fixed because the collection can grow automatically when it becomes full.



- `Vector<Integer> v1 = new Vector<>(5);` // user-defined initial capacity  
`Vector<Integer> v2 = new Vector<>();` // default initial capacity is 10
- So, initial capacity is just the **starting size**, and it can change later as elements are added.

```
import java.util.Vector;

public class Main {
 public static void main(String[] args) {
 Vector<Integer> v = new Vector<>(2, 3);
 // initial capacity = 2, increment = 3

 v.add(10);
 v.add(20);

 System.out.println("Size: " + v.size()); // elementCount = 2
 System.out.println("Capacity: " + v.capacity()); // elementData size = 2

 v.add(30); // exceeds capacity → grows by increment (3)

 System.out.println("After adding one more element:");
 System.out.println("Size: " + v.size()); // elementCount = 3
 System.out.println("Capacity: " + v.capacity()); // new capacity = 5 (2 + 3)
 }
}
```



# The Legacy Methods Defined by **Vector**

| Method                                               | Description                                                                                                       |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>void addElement(E <i>element</i>)</code>       | The object specified by <i>element</i> is added to the vector.                                                    |
| <code>int capacity( )</code>                         | Returns the capacity of the vector.                                                                               |
| <code>Object clone( )</code>                         | Returns a duplicate of the invoking vector.                                                                       |
| <code>boolean contains(Object <i>element</i>)</code> | Returns <b>true</b> if <i>element</i> is contained by the vector, and returns <b>false</b> if it is not.          |
| <code>void copyInto(Object <i>array</i>[ ])</code>   | The elements contained in the invoking vector are copied into the array specified by <i>array</i> .               |
| <code>E elementAt(int <i>index</i>)</code>           | Returns the element at the location specified by <i>index</i> .                                                   |
| <code>Enumeration&lt;E&gt; elements( )</code>        | Returns an enumeration of the elements in the vector.                                                             |
| <code>void ensureCapacity(int <i>size</i>)</code>    | Sets the minimum capacity of the vector to <i>size</i> .                                                          |
| <code>E firstElement( )</code>                       | Returns the first element in the vector.                                                                          |
| <code>int indexOf(Object <i>element</i>)</code>      | Returns the index of the first occurrence of <i>element</i> . If the object is not in the vector, -1 is returned. |

# Stack

**Stack** is a subclass of **Vector** that implements a standard last-in, first-out stack. **Stack** only defines the default constructor, which creates an empty stack. With the release of JDK 5, **Stack** was retrofitted for generics and is declared as:

```
class Stack<E>
```

**E** specifies the type of element stored in the stack.

- **Stack** includes all the methods defined by **Vector** and adds several of its own.

| Method                             | Description                                                                                                                      |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| boolean empty( )                   | Returns <b>true</b> if the stack is empty, and returns <b>false</b> if the stack contains elements.                              |
| E peek( )                          | Returns the element on the top of the stack, but does not remove it.                                                             |
| E pop( )                           | Returns the element on the top of the stack, removing it in the process.                                                         |
| E push(E <i>element</i> )          | Pushes <i>element</i> onto the stack. <i>element</i> is also returned.                                                           |
| int search(Object <i>element</i> ) | Searches for <i>element</i> in the stack. If found, its offset from the top of the stack is returned. Otherwise, -1 is returned. |

- To put an object on the top of the stack, call **push( )**.
- To remove and return the top element, call **pop( )**.
- You can use **peek( )** to return, but not remove, the top object.
- An **EmptyStackException** is thrown if you call **pop( )** or **peek( )** when the invoking stack is empty.
- The **empty( )** method returns **true** if nothing is on the stack.
- The **search( )** method determines whether an object exists on the stack and returns the number of pops that are required to bring it to the top of the stack.
- Example that creates a stack, pushes several **Integer** objects onto it, and then pops them off again:

# Working with Maps

- A *map* is an object that stores associations between keys and values, or *key/value pairs*.
- Given a key, you can find its value. Both keys and values are objects. The keys must be unique, but the values may be duplicated.
- Some maps can accept a **null** key and **null** values, others cannot.
- **Maps don't implement the Iterable interface.** This means that you cannot cycle through a map using a for-each style for loop. Furthermore, you can't obtain an iterator to a map.
- We can obtain a collection-view of a map, which does allow the use of either the for loop or an iterator.

# The Map Interfaces

| Interface    | Description                                                                                  |
|--------------|----------------------------------------------------------------------------------------------|
| Map          | Maps unique keys to values.                                                                  |
| Map.Entry    | Describes an element (a key/value pair) in a map. This is an inner class of <b>Map</b> .     |
| NavigableMap | Extends <b>SortedMap</b> to handle the retrieval of entries based on closest-match searches. |
| SortedMap    | Extends <b>Map</b> so that the keys are maintained in ascending order.                       |

# The Map Interface

- The **Map** interface maps unique keys to values. A *key* is an object that you use to retrieve a value at a later date. Given a key and a value, you can store the value in a **Map** object. After the value is stored, you can retrieve it by using its key.

**Map** is generic and is declared as

```
interface Map<K, V>
```

**K** specifies the type of keys, and **V** specifies the type of values.

- Several methods are declared by **Map**. These methods throw a
- **ClassCastException** -an object is incompatible with the elements in a map.
- A **NullPointerException** - if an attempt is made to use a **null** object and **null** is not allowed in the map.
- An **UnsupportedOperationException** -when an attempt is made to change an unmodifiable map.
- An **IllegalArgumentException**-if an invalid argument is used.

- Maps revolve around two basic operations: **get( )** and **put( )**.
- To put a value into a map, use **put( )**, specifying the key and the value.
- To obtain a value, call **get( )**, passing the key as an argument. The value is returned.
- Maps do not implement the **Collection** interface. However, you can obtain a collection-view of a map.
- To do this, you can use the **entrySet( )** method. It returns a **Set** that contains the elements in the map.
- To obtain a collection-view of the keys, use **keySet( )**.
- To get a collection-view of the values, use **values( )**.
- For all three collectionviews, the collection is backed by the map. Changing one affects the other.
- Collection-views are the means by which maps are integrated into the larger Collections Framework.



```
public class MapDemo {
 public static void main(String[] args) {
 HashMap<Integer, String> map = new HashMap<>();

 map.put(1, "A");
 map.put(2, "B");
 map.put(3, "C");

 // Collection views
 Set<Integer> keys = map.keySet();
 Collection<String> values = map.values();
 Set<Map.Entry<Integer, String>> entries = map.entrySet();

 System.out.println("Keys: " + keys);
 System.out.println("Values: " + values);
 System.out.println("Entries: " + entries);

 // Change via collection view
 keys.remove(2);

 System.out.println("Map after removing key 2: " + map);
 }
}
```

### Output (concept)

```
Keys: [1, 2, 3]
Values: [A, B, C]
Entries: [1=A, 2=B, 3=C]
Map after removing key 2: {1=A, 3=C}
```

The difference is mainly about **what they represent and how they are used**:

## Map

- Stores data as **key-value pairs**
- Example: `{1=A, 2=B}`
- Main methods: `put()`, `get()`, `remove()`
- Not part of `Collection` interface

## Collection View of Map

- Represents **parts of the map as collections**
  - `keySet()` → only keys
  - `values()` → only values
  - `entrySet()` → key-value pairs as objects
- Used for **iteration and bulk operations**
- These are part of the `Collection` framework (Set/Collection)

---

## Key Difference

- **Map** → complete data structure (key + value)
- **Collection view** → partial view of map (keys or values or entries)



# The Map Classes

| Class           | Description                                                                      |
|-----------------|----------------------------------------------------------------------------------|
| AbstractMap     | Implements most of the <b>Map</b> interface.                                     |
| EnumMap         | Extends <b>AbstractMap</b> for use with <b>enum</b> keys.                        |
| HashMap         | Extends <b>AbstractMap</b> to use a hash table.                                  |
| TreeMap         | Extends <b>AbstractMap</b> to use a tree.                                        |
| WeakHashMap     | Extends <b>AbstractMap</b> to use a hash table with weak keys.                   |
| LinkedHashMap   | Extends <b>HashMap</b> to allow insertion-order iterations.                      |
| IdentityHashMap | Extends <b>AbstractMap</b> and uses reference equality when comparing documents. |

| Class           | Order Maintained   | Null Key / Value                 | Thread-Safe | Key Feature / Description                                                        | Declaration Syntax                                                                       |
|-----------------|--------------------|----------------------------------|-------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| HashMap         | No order           | 1 null key, multiple null values | No          | Stores key-value pairs; fast operations using hashing                            | <code>HashMap&lt;Integer, String&gt; map = new HashMap&lt;&gt;();</code>                 |
| LinkedHashMap   | Insertion order    | 1 null key, multiple null values | No          | Maintains insertion order; slightly slower than HashMap                          | <code>LinkedHashMap&lt;Integer, String&gt; map = new LinkedHashMap&lt;&gt;();</code>     |
| TreeMap         | Sorted (ascending) | No null key, allows null values  | No          | Keys stored in sorted order; based on Red-Black Tree                             | <code>TreeMap&lt;Integer, String&gt; map = new TreeMap&lt;&gt;();</code>                 |
| Hashtable       | No order           | No null key, no null value       | Yes         | Legacy class; synchronized (thread-safe)                                         | <code>Hashtable&lt;Integer, String&gt; map = new Hashtable&lt;&gt;();</code>             |
| IdentityHashMap | No order           | Allows null key and values       | No          | Compares keys using <code>==</code> (reference) instead of <code>equals()</code> | <code>IdentityHashMap&lt;Integer, String&gt; map = new IdentityHashMap&lt;&gt;();</code> |
| EnumMap         | Enum order         | No null key (values allowed)     | No          | Keys must be enum type; very fast and memory efficient                           | <code>EnumMap&lt;Day, String&gt; map = new EnumMap&lt;&gt;(Day.class);</code>            |
| WeakHashMap     | No order           | Allows null key and values       | No          | Keys can be garbage collected; used in memory-sensitive applications             | <code>WeakHashMap&lt;Integer, String&gt; map = new WeakHashMap&lt;&gt;();</code>         |

# The HashMap Class

- The **HashMap** class extends **AbstractMap** and implements the **Map** interface.
- It uses a hash table to store the map. It uses a hash table to store the map.
- It does not add any methods of its own.
- Hash map does not guarantee the order of its elements. Therefore, the order in which elements are added to a hash map is not necessarily the order in which they are read by an iterator
- This allows the execution time of **get( )** and **put( )** to remain constant even for large sets.
- **HashMap** is a generic class that has this declaration:

```
class HashMap<K, V>
```

- **K**:The type of keys, and **V** specifies the

Constructors are defined:

- `HashMap( )`
- `HashMap(Map<? extends K, ? extends V> m)`
- `HashMap(int capacity)`
- `HashMap(int capacity, float fillRatio)`type of values.

```
import java.util.*;

class HashMapDemo {
 public static void main(String args[]) {

 // Create a hash map.
 HashMap<String, Double> hm = new HashMap<String, Double>();

 // Put elements to the map
 hm.put("John Doe", new Double(3434.34));
 hm.put("Tom Smith", new Double(123.22));
 hm.put("Jane Baker", new Double(1378.00));
 hm.put("Tod Hall", new Double(99.22));
 hm.put("Ralph Smith", new Double(-19.08));

 // Get a set of the entries.
 Set<Map.Entry<String, Double>> set = hm.entrySet();

 // Display the set.
 for(Map.Entry<String, Double> me : set) {
 System.out.print(me.getKey() + ": ");
 System.out.println(me.getValue());
 }

 System.out.println();

 // Deposit 1000 into John Doe's account.
 double balance = hm.get("John Doe");
 hm.put("John Doe", balance + 1000);
 }
}
```

```
 System.out.println("John Doe's new balance: " +
 hm.get("John Doe"));
 }
}
```

Output from this program is shown here (the precise order may vary):

```
Ralph Smith: -19.08
Tom Smith: 123.22
John Doe: 3434.34
Tod Hall: 99.22
Jane Baker: 1378.0
```

```
John Doe's new balance: 4434.34
```